

# **LAPORAN AKHIR PROJECT BASED LEARNING (PBL)**

**Mata Kuliah Algoritma & Struktur Data (TPL2106)**

Tahun Akademik 2025/2026

## **SISTEM PENGELOLAAN LAPORAN DAN NILAI MAHASISWA BERBASIS COMMAND LINE INTERFACE (CLI)**

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

**Disusun oleh:**

| <b>Nama</b>                           | <b>NIM</b>         |
|---------------------------------------|--------------------|
| <b>Alberto Relva Ezekiel Garnedy</b>  | <b>J0403251090</b> |
| <b>Caesarian Lanang Parizano Zaim</b> | <b>J0403251016</b> |
| <b>Euwlipin Natanoel Saudale</b>      | <b>J0403251121</b> |

**Program Studi Teknologi Rekayasa Perangkat Lunak**

**Sekolah Vokasi IPB University**

**2026**



## KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan akhir Project Based Learning (PBL) untuk mata kuliah Algoritma & Struktur Data (TPL2106) ini dapat diselesaikan dengan baik. Proyek ini merupakan sarana bagi penulis untuk menerapkan konsep-konsep struktur data teoretis ke dalam sebuah solusi perangkat lunak nyata.

Penulis menyampaikan terima kasih yang sebesar-besarnya kepada Ibu Dr. Shelve Nidya Neyman selaku dosen pengampu mata kuliah Algoritma & Struktur Data yang telah memberikan bimbingan, arahan, dan ilmu pengetahuannya selama proses perkuliahan dan penyusunan proyek ini. Terima kasih juga diucapkan kepada semua pihak yang telah mendukung kelancaran proyek ini.

Penulis menyadari bahwa aplikasi dan laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi pengembangan lebih lanjut di masa mendatang.

Depok,

Juni

2026

Penulis

## DAFTAR ISI

|                                                       |    |
|-------------------------------------------------------|----|
| BAB I. PENDAHULUAN.....                               | 1  |
| 1.1 Latar Belakang.....                               | 1  |
| 1.2 Tujuan Proyek.....                                | 1  |
| 1.3 Ruang Lingkup.....                                | 1  |
| BAB II. ANALISIS DAN PERANCANGAN.....                 | 3  |
| 2.1 Deskripsi Sistem .....                            | 3  |
| 2.2 Struktur Data yang Digunakan.....                 | 3  |
| 2.3 Algoritma yang Digunakan.....                     | 4  |
| 1) Searching .....                                    | 4  |
| 2) Sorting .....                                      | 4  |
| 3) Manajemen State & Logika Struktur Data (ADT) ..... | 5  |
| 4) Algoritma Validasi dan Pembersihan Teks .....      | 5  |
| 5) Algoritma Rekursif untuk Kalkulasi Data .....      | 5  |
| 2.4 Diagram Alir Program (Flowchart) .....            | 6  |
| 2.5 Struktur Program.....                             | 7  |
| BAB III. IMPLEMENTASI PROGRAM.....                    | 9  |
| 3.1 Tampilan Menu Utama .....                         | 9  |
| 3.2 Fitur Create (Add Student).....                   | 9  |
| 3.3 Fitur Read (Show Students).....                   | 9  |
| 3.4 Fitur Update .....                                | 10 |
| 3.5 Fitur Delete .....                                | 10 |
| 3.6 Fitur Searching.....                              | 11 |
| 3.7 Fitur Sorting.....                                | 11 |
| 3.8 Penyimpanan Data (File Handling) .....            | 12 |

|                                            |    |
|--------------------------------------------|----|
| 3.9 Validasi Input dan Error Handling..... | 13 |
| BAB IV. PENGUJIAN DAN ANALISIS .....       | 14 |
| 4.1 Skenario Pengujian .....               | 14 |
| 4.2 Kelebihan Sistem .....                 | 14 |
| 4.3 Keterbatasan Sistem.....               | 15 |
| BAB V. KENDALA DAN SOLUSI .....            | 16 |
| BAB VI. PEMBAGIAN TUGAS ANGGOTA .....      | 16 |
| BAB VII. KESIMPULAN DAN SARAN.....         | 17 |
| 7.1 Kesimpulan .....                       | 17 |
| 7.2 Saran .....                            | 17 |
| DAFTAR PUSTAKA .....                       | 18 |
| LAMPIRAN.....                              | 18 |

# **BAB I. PENDAHULUAN**

## **1.1 Latar Belakang**

Pengelolaan data akademis mahasiswa, khususnya nilai mata kuliah, merupakan salah satu elemen krusial dalam administrasi institusi pendidikan. Selama ini, rekapitulasi data sering menghadapi kendala akurasi, risiko duplikasi, serta lambatnya proses analisis statistik seperti penentuan nilai rata-rata, nilai tertinggi, nilai terendah, dan pemeringkatan mahasiswa.

Proyek ini dibuat untuk membantu pengelolaan data siswa dan nilai mata kuliah secara lebih terstruktur, efisien, dan aman dari kesalahan input. Dengan membangun sistem berbasis Terminal/Command Line Interface (CLI) menggunakan bahasa pemrograman Python, sistem ini mengimplementasikan berbagai macam struktur data seperti Linked List, Queue, Stack, dan Set, serta algoritma pencarian dan pengurutan tingkat lanjut untuk menjamin performa dan keandalan sistem dalam mengolah data.

## **1.2 Tujuan Proyek**

Tujuan dari pelaksanaan proyek ini adalah sebagai berikut:

1. Membuat aplikasi pengelolaan data laporan nilai mahasiswa berbasis terminal menggunakan Python.
2. Mengimplementasikan berbagai macam struktur data (Linked List, Queue, Stack, Set, Tuple, Dictionary, dan List) sesuai dengan karakteristik kebutuhan data.
3. Menerapkan algoritma pengurutan Merge Sort dan pencarian Binary Search serta teknik rekursi dalam analisis nilai.
4. Menerapkan konsep file handling berbasis JSON agar data tersimpan secara permanen dan terstruktur.
5. Mengembangkan solusi validasi data yang ketat untuk mencegah inkonsistensi nama, format angka, dan data duplikat.

## **1.3 Ruang Lingkup**

Ruang lingkup dari sistem ini mencakup:

- Platform aplikasi berjalan pada CLI (Console) menggunakan bahasa Python.

- Penyimpanan data secara permanen menggunakan format file JSON (studentData.json).
- Menyediakan fitur CRUD (Create, Read, Update, Delete) data siswa dan nilainya.
- Menyediakan fitur searching (pencarian parsial dan presisi) serta sorting (Merge Sort berdasarkan rata-rata nilai).
- Menyediakan mekanisme pembatalan aksi (Undo) berbasis struktur data Stack.
- Ekspor laporan komprehensif ke format berkas PDF (student\_report.pdf).

## BAB II. ANALISIS DAN PERANCANGAN

### 2.1 Deskripsi Sistem

Sistem Report Mahasiswa ini memungkinkan pengguna (administrator atau dosen) untuk mengelola data nama mahasiswa dan lima slot nilai subjek secara dinamis. Sistem ini memisahkan antara data mentah yang dimasukkan dan data valid yang siap diproses. Sebelum data dapat disimpan ke file JSON atau diekspor ke PDF, pengguna diwajibkan menjalankan modul pemrosesan data (Process Data) untuk memvalidasi kelayakan data dari aspek duplikasi, karakter nama ilegal, serta jangkauan nilai subjek (0-100).

### 2.2 Struktur Data yang Digunakan

| Struktur Data | Fungsi & Deskripsi Detail                                                                                                                                                                                                |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linked List   | Diimplementasikan melalui kelas StudentNode dan StudentLinkedList. Digunakan untuk menampung data utama mahasiswa dari memori atau berkas saat pembacaan dan proses iterasi agar manajemen memori dinamis lebih optimal. |
| Queue         | Diimplementasikan pada kelas Queue dengan optimasi pembersihan memori indeks depan. Digunakan untuk mengelola antrean objek mahasiswa yang akan divalidasi satu per satu pada fungsi processData().                      |
| Stack         | Diimplementasikan melalui kelas Stack berbasis list. Digunakan untuk menyimpan riwayat state data (deep copy) guna memfasilitasi fitur 'Undo Last Action' hingga langkah awal.                                           |
| Dictionary    | Digunakan untuk merepresentasikan satu entitas mahasiswa yang memuat pasangan kunci-nilai seperti nama, array nilai, dan                                                                                                 |



|       |                                                                                                                                                                |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | statistik penunjang.                                                                                                                                           |
| List  | Digunakan untuk menampung koleksi utama studentData, validStudentData, serta koleksi nilai individual subjek.                                                  |
| Set   | Digunakan dalam fungsi processData() sebagai penampung nama mahasiswa terformat (seenNames) untuk mendeteksi adanya duplikasi nama mahasiswa secara real-time. |
| Tuple | Digunakan untuk mendefinisikan konstanta subjects (Subject 1 sampai Subject 5) yang bersifat immutable (tidak dapat diubah) selama program berjalan.           |

#### Alasan Pemilihan Struktur Data:

Queue dipilih karena mengikuti prinsip First-In-First-Out (FIFO) yang sangat adil dan terstruktur untuk memproses antrean validasi data mahasiswa. Stack dipilih karena mengikuti prinsip Last-In-First-Out (LIFO), yang secara natural menyimpan kondisi data terakhir untuk dikembalikan saat pengguna membatalkan aksi. Linked List memberikan fleksibilitas alokasi memori dinamis tanpa perlu mendefinisikan kapasitas awal data.

## 2.3 Algoritma yang Digunakan

### 1) Searching

Sistem mengimplementasikan dua jenis pencarian nama: Binary Search secara rekursif (recursiveBinarySearch) untuk mencari data yang sudah terurut secara presisi, serta Partial Name Search (recursivePartialNameSearch) secara rekursif untuk mengumpulkan semua data mahasiswa yang mengandung keyword tertentu secara case-insensitive.

### 2) Sorting

Sistem menggunakan Merge Sort (mergeSort) dengan pendekatan Divide and Conquer. Algoritma ini membagi list data menjadi sub-list secara rekursif, mengurutkannya, dan

menggabungkannya kembali. Alasan penggunaan Merge Sort adalah karena memiliki efisiensi waktu yang konsisten, yaitu  $O(N \log N)$  baik dalam kondisi terbaik, rata-rata, maupun terburuk, serta stabil (stable sort) saat mengurutkan nilai rata-rata mahasiswa.

### **3) Manajemen State & Logika Struktur Data (ADT)**

Program ini mengimplementasikan struktur data Stack pada fitur *Undo* (Menu 11) berbasis prinsip LIFO lewat metode append dan pop untuk menduplikasi serta memulihkan *state* data ke kondisi sebelumnya demi menjamin keamanan manipulasi data, Queue pada pemrosesan data (Menu 5) dengan prinsip FIFO dibantu algoritma *compaction* memori guna menyisir antrean data dari depan secara adil sekaligus mencegah kebocoran memori (*memory leak*), serta Linked List (Menu 2 & 5) yang menggunakan referensi pointer `current.next` untuk menelusuri data siswa secara berantai dari node kepala guna mempermudah alokasi memori dinamis dan menghindari ketergantungan pada indeks array konvensional yang kaku.

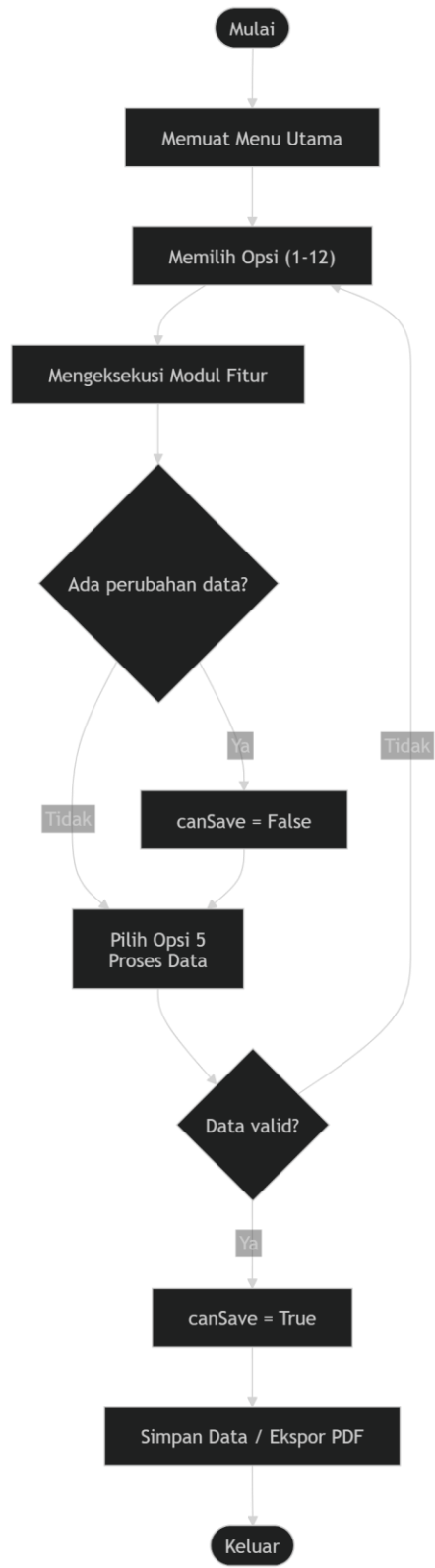
### **4) Algoritma Validasi dan Pembersihan Teks**

Untuk sanitasi data pada Menu 5, program menerapkan algoritma Normalisasi Nama yang menggunakan kombinasi fungsi `.lower()`, `.strip()`, dan `.join(text.split())` untuk mengubah teks menjadi huruf kecil serta menghapus spasi ganda agar sistem dapat mengidentifikasi kesamaan teks dan menghindari duplikasi nama terselubung, serta Validasi Karakter lewat fungsi bawaan `all()` yang menyisir string karakter demi karakter untuk memastikan setiap input nama hanya berisi huruf alfabet atau tanda baca standar sehingga basis data lokal (`studentData.json`) tetap steril dari simbol ilegal atau angka.

### **5) Algoritma Rekursif untuk Kalkulasi Data**

Pada kalkulasi statistik siswa di Menu 5, program menghindari perulangan iteratif tradisional dengan menggunakan fungsi rekursif `recursiveSum` untuk menjumlahkan nilai subjek, `recursiveMinMax` untuk mencari batas nilai terkecil dan terbesar sekaligus, serta `recursiveInvalidGrades` untuk menyaring data nilai yang melanggar aturan \$0 - 100\$, di mana ketiganya bekerja dengan memanfaatkan tumpukan eksekusi memori (call stack) yang memproses elemen secara berantai hingga mencapai batas akhir agar kode program menjadi lebih modular, fungsional, dan meminimalkan risiko mutabilitas (perubahan) data asli secara tidak sengaja.

## 2.4 Diagram Alir Program (Flowchart)



## 2.5 Struktur Program

Aplikasi ini disusun dalam kesatuan file skrip terstruktur:

- `student_report_crud.py` (File Program Utama)

Berkas ini merupakan skrip Python utama yang mendefinisikan seluruh logika bisnis, alur menu interaktif, serta implementasi struktur data dinamis di dalam memori runtime (RAM). Fungsinya adalah untuk mengeksekusi operasi manipulasi data siswa secara langsung menggunakan struktur data Linked List untuk penelusuran objek secara berantai, Stack untuk memfasilitasi fitur pembatalan aksi (Undo) berdasarkan prinsip LIFO, serta Queue untuk mengelola antrean pemrosesan statistik nilai secara adil berdasarkan prinsip FIFO.

- `studentData.json` (Basis Data Lokal)

Berkas ini adalah media penyimpanan berbasis teks dengan format standar JSON (JavaScript Object Notation) yang menyusun informasi dalam bentuk kumpulan objek terstruktur (Array of Objects). Fungsinya bertindak sebagai basis data lokal permanen (persistence storage) untuk menjaga agar seluruh data mentah—berupa nama lengkap beserta susunan nilai subjek siswa—tetap tersimpan dengan aman dan tidak hilang ketika program Python ditutup atau komputer dimatikan.

- `student_report.pdf` (Laporan Hasil Pengolahan)

Berkas ini merupakan dokumen cetak digital statis berformat PDF yang diterbitkan secara otomatis oleh sistem dengan memanfaatkan bantuan pustaka (library) FPDF. Fungsinya adalah untuk menyajikan visualisasi data sekunder hasil kalkulasi algoritma rekursif (seperti nilai rata-rata, nilai tertinggi, dan nilai terendah) yang telah diatur rapi dalam bentuk tabel peringkat (ranking) setelah melewati proses pengurutan menggunakan algoritma Merge Sort.

- `student_report_crud.cpython-313.pyc` (Bytecode Terkompilasi)

Berkas biner ini merupakan hasil kompilasi otomatis (compiled bytecode) yang dihasilkan oleh interpreter Python versi 3.13 dan disimpan di dalam direktori khusus bernama `__pycache__`. Fungsinya adalah untuk meningkatkan efisiensi dan mempercepat waktu pemuatan (loading time) saat program dijalankan kembali, karena mesin komputer dapat

langsung membaca instruksi tingkat rendah tanpa harus menerjemahkan ulang kode teks skrip Python dari awal.

- `.gitattributes` (Konfigurasi Git)

Berkas ini adalah file konfigurasi teks sederhana yang digunakan secara khusus oleh sistem pengontrol versi (version control system) Git untuk mengatur properti repositori proyek. Fungsinya adalah untuk mendeteksi jenis berkas teks secara otomatis dan melakukan normalisasi karakter akhir baris (line endings normalization), sehingga baris kode program Anda tetap konsisten, rapi, dan tidak mengalami kerusakan struktural ketika dikembangkan lintas sistem operasi seperti Windows, Linux, atau Mac.

## BAB III. IMPLEMENTASI PROGRAM

### 3.1 Tampilan Menu Utama

```
===== STUDENT REPORT SYSTEM =====
1. Add Student
2. Show Students
3. Update Student
4. Delete Student
5. Process Data
6. Show Ranking
7. Save File
8. Load File
9. Export PDF
10. Search Student
11. Undo Last Change
12. Exit

Choose: █
```

### 3.2 Fitur Create (Add Student)

Fungsi `addStudent()` meminta input nama mahasiswa dan memanggil `inputGrades()` untuk mengulang pengisian nilai subjek 1-5. Sebelum data ditambahkan ke dalam list `studentData`, kondisi data sebelumnya disimpan ke dalam `undoStack` melalui `recordUndo()`.

```
Choose: 1
Student Full Name: Alberto Depok
Alberto Depok Subject 1 Grade: 100
Alberto Depok Subject 2 Grade: 100
Alberto Depok Subject 3 Grade: 100
Alberto Depok Subject 4 Grade: 100
Alberto Depok Subject 5 Grade: 100
Student Added
```

### 3.3 Fitur Read (Show Students)

Fungsi `showStudents()` mengonversi data list menjadi objek `StudentLinkedList` melalui method `dariList()`. Struktur kemudian ditelusuri dengan generator (`__iter__`) untuk mencetak nama mahasiswa beserta daftar nilainya secara terstruktur.

```
Choose: 2

===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 100, 100]
```

### 3.4 Fitur Update

Fungsi `updateStudent()` memberikan opsi pembaruan sub-menu yang sangat fleksibel: (1) Mengubah nama saja, (2) Mengubah nilai satu subjek tertentu, (3) Menghapus/mengnolkan nilai satu subjek, atau (4) Mengganti seluruh nilai subjek sekaligus.

```
Choose: 3

===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 100, 100]
Select student index to update: 1

Selected Student: Alberto Depok

===== UPDATE MENU =====
1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade
4. Replace All Grades
5. Cancel

Choose update option: 2

===== SUBJECT GRADES =====
1. Subject 1: 100
2. Subject 2: 100
3. Subject 3: 100
4. Subject 4: 100
5. Subject 5: 100
Select subject index to update: 4
New Subject 4 Grade: 75
Subject Grade Updated
```

### 3.5 Fitur Delete

Fungsi `deleteStudent()` menampilkan daftar mahasiswa, meminta indeks target berbasis angka (diubah ke indeks pemrograman internal), melakukan pencatatan undo, dan membuang elemen menggunakan metode `.pop(index)`.

```
Choose: 4

===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 75, 100]
Select student index to delete: 1
Student Deleted
```

### 3.6 Fitur Searching

Fungsi `searchStudent()` menggunakan keyword yang dimasukkan pengguna. Data mahasiswa diurutkan terlebih dahulu berdasarkan nama, kemudian dilakukan pencarian parsial secara rekursif. Jika ditemukan satu kecocokan, detail nilai dan kalkulasi statistik langsung ditampilkan.

```
Choose: 10
Search student name keyword: alb

===== STUDENT DETAIL =====
Name   : Alberto Depok
Subject 1: 100
Subject 2: 100
Subject 3: 100
Subject 4: 75
Subject 5: 100
```

### 3.7 Fitur Sorting

Pengurutan dilakukan pada fungsi `showRankings()` dan `exportPDF()` dengan memanggil `mergeSort()` secara descending (`reverse=True`) berdasarkan key rata-rata nilai (`student['avg']`) mahasiswa untuk menghasilkan susunan peringkat 1 hingga N.

```
===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak - [100, 100, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
```

```
Choose: 6

===== STUDENT RANKING =====
Rank 1 | Bro Satu | AVG: 100.00 | MAX: 100 | MIN: 100
Rank 2 | Alberto Depok | AVG: 95.00 | MAX: 100 | MIN: 75
Rank 3 | Zoltraak | AVG: 93.00 | MAX: 100 | MIN: 65
Rank 4 | klingon zor | AVG: 55.00 | MAX: 55 | MIN: 55
```



### 3.8 Penyimpanan Data (File Handling)

Fungsi `saveFile()` memanfaatkan pustaka standard 'json' untuk membuang seluruh isi struktur data `studentData` ke dalam berkas '`studentData.json`' dengan indentasi terformat agar mudah dibaca manusia. Fungsi `loadFile()` membaca berkas tersebut kembali ke memori.

```
studentData.json x
studentData.json > ...
1  [
2    {
3      "name": "Alberto Depok",
4      "grades": [
5        100,
6        100,
7        100,
8        75,
9        100
10     ]
11   },
12   {
13     "name": "Bro Satu",
14     "grades": [
15       100,
16       100,
17       100,
18       100,
19       100
20     ]
21   },
22   {
23     "name": "Zoltraak",
24     "grades": [
25       100,
26       100,
27       100,
28       100,
29       65
30     ]
31   }
32 ]
```

```
studentData.json x
studentData.json > ...
12  {
13    "name": "Bro Satu",
14    "grades": [
15      100,
16      100,
17      100,
18      100,
19      100
20    ]
21  },
22  {
23    "name": "Zoltraak",
24    "grades": [
25      100,
26      100,
27      100,
28      100,
29      65
30    ]
31  },
32  {
33    "name": "klinton zor",
34    "grades": [
35      55,
36      55,
37      55,
38      55,
39      55
40    ]
41  }
42 ]
```

### **3.9 Validasi Input dan Error Handling**

Sistem mengimplementasikan validasi berlapis:

1. Nilai numerik harus berupa integer dan divalidasi menggunakan try-except ValueError pada fungsi inputGrade().
2. Nama mahasiswa divalidasi dengan isValidName() menggunakan all(char.isalpha() or char in " !-" for char in cleanName).
3. Jangkauan nilai wajib berada pada rentang 0-100 yang divalidasi secara rekursif oleh fungsi recursiveInvalidGrades().

## BAB IV. PENGUJIAN DAN ANALISIS

### 4.1 Skenario Pengujian

| No | Skenario Pengujian                                                      | Hasil / Respon Sistem                                                                               |
|----|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1  | Menambahkan data siswa baru dengan nama 'Andi' dan nilai valid.         | Berhasil ditambahkan, status canSave diset menjadi False (perlu diproses kembali).                  |
| 2  | Memasukkan input huruf pada kolom pengisian nilai subjek.               | Ditangkap oleh penanganan ValueError, menampilkan 'Grade must be a number' dan meminta input ulang. |
| 3  | Menjalankan Process Data dengan data yang memiliki nama duplikat.       | Menampilkan pesan error 'Duplicate Name: [Nama]' dan memblokir fitur penyimpanan berkas.            |
| 4  | Menampilkan menu peringatan (Show Ranking) sebelum data diproses.       | Menampilkan peringatan 'Process Data Before Show Rankings'.                                         |
| 5  | Melakukan aksi pencarian kata kunci nama parsial 'An'.                  | Berhasil mengembalikan daftar mahasiswa yang mengandung suku kata 'An'.                             |
| 6  | Mengeksekusi fitur Undo Last Change setelah melakukan penghapusan data. | Data mahasiswa yang terhapus berhasil dikembalikan secara utuh dari Stack.                          |

### 4.2 Kelebihan Sistem

1. Struktur data yang variatif dan tersegregasi membuat manipulasi memori terstruktur.
2. Validasi data yang ketat menjamin akurasi data nilai sebelum disimpan secara permanen.
3. Adanya fitur Undo memberikan kenyamanan bagi pengguna apabila salah melakukan operasi CRUD.

### **4.3 Keterbatasan Sistem**

1. Antarmuka masih berbasis teks (Command Line Interface).
2. Proses sorting dan validasi dilakukan secara menyeluruh di memori utama, belum menggunakan basis data transaksional.

## BAB V. KENDALA DAN SOLUSI

| Kendala                                                                         | Solusi                                                                                                                      |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Inkonsistensi data nilai (angka minus atau melebihi 100) dan nama kosong.       | Dibuat fungsi rekursif <code>recursiveInvalidGrades()</code> untuk menyaring data ilegal saat fase pemrosesan.              |
| Perubahan data yang tidak sengaja merusak struktur data di memori.              | Mengimplementasikan Stack penampung state dengan <code>copy.deepcopy()</code> sehingga data lama tersimpan mandiri.         |
| Penumpukan memori indeks depan pada Queue akibat operasi <code>dequeue</code> . | Menambahkan logika penciutan internal (compaction) pada kelas Queue jika indeks depan telah melewati ambang batas tertentu. |

## BAB VI. PEMBAGIAN TUGAS ANGGOTA

### Nama

### Tugas

Alberto Relva Ezekiel Garnedy

Membuat kelas `StudentNode`, `StudentLinkedList` (untuk iterasi data utama), kelas `Stack`, serta fungsi `recordUndo` dan `undoLastAction` (Menu 11).

Caesarian Lanang Parizano Zaim

Membuat kelas `Queue` (dengan optimasi indeks), fungsi `processData` (Menu 5) untuk validasi data lewat antrian dan `Set`, serta fungsi file handling (`saveFile`, `loadFile`, `exportPDF`) (Menu 7, 8, 9).

Euwlipin Natanoel Saudale

Membuat antarmuka fungsi menu (Menu 12), fungsi dasar CRUD (`addStudent`, `showStudents`, `updateStudent`, `deleteStudent`) (Menu 1-4), serta fungsi rekursif untuk algoritma pengurutan (`mergeSort`) (Menu 6) dan pencarian (`recursiveBinarySearch`, `recursivePartialNameSearch`) (Menu 10).

## **BAB VII. KESIMPULAN DAN SARAN**

### **7.1 Kesimpulan**

1. Aplikasi pengelolaan laporan nilai mahasiswa berhasil dibangun berbasis CLI dengan mengintegrasikan struktur data dinamis secara harmonis.
2. Penerapan algoritma pengurutan Merge Sort terbukti efektif dalam menyusun urutan peringkat mahasiswa berdasarkan nilai rata-rata secara akurat.
3. Fitur pengolahan antrean (Queue) dan pembatalan (Stack Undo) bekerja sesuai dengan prinsip teori dasar algoritma dan struktur data.

### **7.2 Saran**

1. Mengembangkan aplikasi ke dalam bentuk Graphical User Interface (GUI) atau berbasis web agar lebih mudah digunakan.
2. Mengintegrasikan sistem dengan database relasional seperti SQLite atau PostgreSQL untuk retensi data skala besar.

## DAFTAR PUSTAKA

*Algoritma dan Struktur Data Menggunakan Pemogramman Python*. (t.t.). Diambil 26 Juni 2026,

dari <https://repository.penerbiteureka.com/publications/593973/algoritma-dan-struktur-data-menggunakan-pemogramman-python>

Pandiangan, G. R., Naufal Aqiilah Asra, Mohd. Rafiif Albani, Angelica Barus, & Adidtya

Perdana. (2026). Analisis Komparatif Efisiensi Merge Sort dan Quick Sort Menggunakan Pendekatan Divide and Conquer pada Berbagai Kondisi Data Produk Sistem E-Commerce. *MUARA KOMPUTER : Jurnal Ilmiah Ilmu Komputer & Elektronika*, 2(2), 1–15. <https://doi.org/10.64365/murakom.v2i2.278>

## LAMPIRAN

### Lampiran A. Link GitHub

[https://github.com/Brurur/ASD\\_A\\_1](https://github.com/Brurur/ASD_A_1)

### Lampiran B. Screenshot Program

```
===== STUDENT REPORT SYSTEM =====
1. Add Student
2. Show Students
3. Update Student
4. Delete Student
5. Process Data
6. Show Ranking
7. Save File
8. Load File
9. Export PDF
10. Search Student
11. Undo Last Change
12. Exit

Choose: █
```

```
Choose: 1
Student Full Name: Solomon Grundy
Solomon Grundy Subject 1 Grade: 76
Solomon Grundy Subject 2 Grade: 35
Solomon Grundy Subject 3 Grade: 87
Solomon Grundy Subject 4 Grade: 67
Solomon Grundy Subject 5 Grade: 98
Student Added
```

```
Choose: 2

===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak - [100, 100, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]
```

```
Choose: 3

===== ALL STUDENT LIST =====
1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak - [100, 100, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]
Select student index to update: 3

Selected Student: Zoltraak

===== UPDATE MENU =====
1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade
4. Replace All Grades
5. Cancel

Choose update option: 1
New Name: Zoltraak Rumber
Student Name Updated
```



Choose: 3

===== ALL STUDENT LIST =====

1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak Rumber - [100, 100, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]

Select student index to update: 3

Selected Student: Zoltraak Rumber

===== UPDATE MENU =====

1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade
4. Replace All Grades
5. Cancel

Choose update option: 2

===== SUBJECT GRADES =====

1. Subject 1: 100
2. Subject 2: 100
3. Subject 3: 100
4. Subject 4: 100
5. Subject 5: 65

Select subject index to update: 1

New Subject 1 Grade: 87

Subject Grade Updated

Choose: 3

===== ALL STUDENT LIST =====

1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak Rumber - [87, 100, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]

Select student index to update: 3

Selected Student: Zoltraak Rumber

===== UPDATE MENU =====

1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade
4. Replace All Grades
5. Cancel

Choose update option: 3

===== SUBJECT GRADES =====

1. Subject 1: 87
2. Subject 2: 100
3. Subject 3: 100
4. Subject 4: 100
5. Subject 5: 65

Select subject index to clear: 2

Subject Grade Cleared

Choose: 3

===== ALL STUDENT LIST =====

1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak Rumber - [87, 0, 100, 100, 65]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]

Select student index to update: 3

Selected Student: Zoltraak Rumber

===== UPDATE MENU =====

1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade
4. Replace All Grades
5. Cancel

Choose update option: 4

Zoltraak Rumber Subject 1 Grade: 100

Zoltraak Rumber Subject 2 Grade: 100

Zoltraak Rumber Subject 3 Grade: 100

Zoltraak Rumber Subject 4 Grade: 100

Zoltraak Rumber Subject 5 Grade: 100

All Grades Updated

Choose: 4

===== ALL STUDENT LIST =====

1. Alberto Depok - [100, 100, 100, 75, 100]
2. Bro Satu - [100, 100, 100, 100, 100]
3. Zoltraak Rumber - [100, 100, 100, 100, 100]
4. klingon zor - [55, 55, 55, 55, 55]
5. Solomon Grundy - [76, 35, 87, 67, 98]

Select student index to delete: 5

Student Deleted

Choose: 5

Data Has Been Processed!

Choose: 6

===== STUDENT RANKING =====

Rank 1 | Bro Satu | AVG: 100.00 | MAX: 100 | MIN: 100

Rank 2 | Zoltraak Rumber | AVG: 100.00 | MAX: 100 | MIN: 100

Rank 3 | Alberto Depok | AVG: 95.00 | MAX: 100 | MIN: 75

Rank 4 | klingon zor | AVG: 55.00 | MAX: 55 | MIN: 55

```
Choose: 7  
Data Saved
```

```
Choose: 8  
Data Loaded
```

```
Choose: 9  
Process Data Before Exporting
```

```
Choose: 9  
PDF exported!
```

```
Choose: 10  
Search student name keyword: z  
  
===== MATCHING STUDENTS =====  
1. klingon zor - [55, 55, 55, 55, 55]  
2. Zoltraak Rumber - [100, 100, 100, 100, 100]  
Select result index to show detail: 2  
  
===== STUDENT DETAIL =====  
Name   : Zoltraak Rumber  
Subject 1: 100  
Subject 2: 100  
Subject 3: 100  
Subject 4: 100  
Subject 5: 100  
AVG    : 100.00  
MAX    : 100  
MIN    : 100
```

```
Choose: 11  
Undo Success: Load File
```

## Lampiran C. Struktur Folder Proyek

```
> __pycache__  
◆ .gitattributes  
📁 student_report_cr... 1  
📄 student_report.pdf  
{} studentData.json
```

Lampiran D. Commit History GitHub

Brurur / ASD\_A.1Public

NotificationsFork0Star0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Commits

main

All usersAll time

Commits on Jun 26, 2026

Create README.md for Student Report SystemVerified437c257LipiNoeL authored 1 hour ago

Commits on Jun 3, 2026

update pjblfffb741LipiNoeL committed 3 weeks ago

Commits on Jun 2, 2026

update pjbl asd5d6d9dcLipiNoeL committed 3 weeks ago

Commits on Apr 17, 2026

Fitur PDF3be11a8Brurur committed on Apr 17

update pjblfffb741LipiNoeL committed 3 weeks ago

Commits on Jun 2, 2026

update pjbl asd5d6d9dcLipiNoeL committed 3 weeks ago

Commits on Apr 17, 2026

Fitur PDF3be11a8Brurur committed on Apr 17

Commits on Apr 16, 2026

push proyek newd8d22b0LipiNoeL committed on Apr 16

Commits on Feb 27, 2026

Initial commit143d1aeBrurur committed on Feb 27

© 2026 GitHub, Inc. TermsPrivacySecurityStatusCommunityDocsContactManage cookiesDo not share my personal information

Lampiran E. Source Code Utama

```
import copy
import json

try:
    from fpdf import FPDF
except ImportError:
```

```

FPDF = None

# =====
# COLLECTION DATA TYPES
# =====
# list : studentData, validStudentData, grades
# dict : student object
# tuple : SUBJECTS
# set : duplicate-name validation
SUBJECTS = tuple(f"Subject {index}" for index in range(1, 6))
DATA_FILE = "studentData.json"
REPORT_FILE = "student_report.pdf"

# =====
# LINKED LIST
# =====
class StudentNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class StudentLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.length = 0

    def append(self, data):
        node = StudentNode(data)

        if self.head is None:
            self.head = node
            self.tail = node
        else:
            self.tail.next = node
            self.tail = node

        self.length += 1

    def __iter__(self):
        current = self.head

```

```

        while current is not None:
            yield current.data
            current = current.next

    @classmethod
    def fromList(cls, data):
        linkedList = cls()

        for item in data:
            linkedList.append(item)

        return linkedList

# =====
# STACK
# =====
class Stack:
    def __init__(self):
        self.items = list()

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.isEmpty():
            return None

        return self.items.pop()

    def isEmpty(self):
        return len(self.items) == 0

# =====
# QUEUE
# =====
class Queue:
    def __init__(self):
        self.items = list()
        self.frontIndex = 0

    def enqueue(self, item):
        self.items.append(item)

```

```

def dequeue(self):
    if self.isEmpty():
        return None

    item = self.items[self.frontIndex]
    self.frontIndex += 1

    # Compact the queue occasionally so memory stays tidy for large data.
    if self.frontIndex > 50 and self.frontIndex * 2 >= len(self.items):
        self.items = self.items[self.frontIndex:]
        self.frontIndex = 0

    return item

def isEmpty(self):
    return self.frontIndex >= len(self.items)

studentData = list()
validStudentData = list()
undoStack = Stack()
canSave = True

# =====
# INPUT HELPERS
# =====
def inputGrade(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Grade must be a number.")

def inputStudentIndex(prompt):
    try:
        return int(input(prompt)) - 1
    except ValueError:
        return -1

def inputGrades(name):
    grades = list()

    for subject in SUBJECTS:

```



```

        grade = inputGrade(f"{name} {subject} Grade: ")
        grades.append(grade)

    return grades

def showSubjectGrades(student):
    grades = student.get("grades", list())

    print("\n===== SUBJECT GRADES =====")

    for index, subject in enumerate(SUBJECTS, start=1):
        gradeIndex = index - 1

        if gradeIndex < len(grades):
            grade = grades[gradeIndex]
        else:
            grade = "-"

        print(f"{index}. {subject}: {grade}")

def ensureGradeSlots(student):
    grades = student.setdefault("grades", list())

    while len(grades) < len(SUBJECTS):
        grades.append(0)

    return grades

def markDataChanged():
    global canSave
    canSave = False
    validStudentData.clear()

def recordUndo(action):
    undoStack.push({
        "action": action,
        "studentData": copy.deepcopy(studentData),
        "validStudentData": copy.deepcopy(validStudentData),
        "canSave": canSave
    })

```

```

def normalizeName(name):
    return " ".join(str(name).strip().lower().split())

def isValidName(name):
    cleanName = str(name).strip()

    if cleanName == "":
        return False

    return all(char.isalpha() or char in " .'-" for char in cleanName)

def findProcessedStudentByName(name):
    targetName = normalizeName(name)

    for student in validStudentData:
        if normalizeName(student["name"]) == targetName:
            return student

    return None

def showStudentDetail(student):
    processedStudent = findProcessedStudentByName(student["name"])

    print("\n==== STUDENT DETAIL =====")
    print(f"Name      : {student['name']}")

    for index, subject in enumerate(SUBJECTS):
        grades = student.get("grades", list())

        if index < len(grades):
            grade = grades[index]
        else:
            grade = "-"

        print(f"{subject}: {grade}")

    if processedStudent is not None:
        print(f"AVG      : {processedStudent['avg']:.2f}")
        print(f"MAX      : {processedStudent['max']}")
        print(f"MIN      : {processedStudent['min']}")

```

```
# =====
```

```

# RECURSIVE HELPERS
# =====
def recursiveSum(values, index=0):
    if index == len(values):
        return 0

    return values[index] + recursiveSum(values, index + 1)

def recursiveMinMax(values, index=0, currentMin=None, currentMax=None):
    if index == len(values):
        return currentMin, currentMax

    value = values[index]

    if currentMin is None or value < currentMin:
        currentMin = value

    if currentMax is None or value > currentMax:
        currentMax = value

    return recursiveMinMax(values, index + 1, currentMin, currentMax)

def recursiveInvalidGrades(grades, index=0, invalidGrades=None):
    if invalidGrades is None:
        invalidGrades = list()

    if index == len(grades):
        return invalidGrades

    grade = grades[index]

    if not isinstance(grade, int) or not 0 <= grade <= 100:
        invalidGrades.append((index, grade))

    return recursiveInvalidGrades(grades, index + 1, invalidGrades)

# =====
# SORTING & SEARCHING
# =====
def sortValue(item, key):
    value = key(item)

```

```

    if isinstance(value, str):
        return value.lower()

    return value

def merge(left, right, key, reverse):
    result = list()
    leftIndex = 0
    rightIndex = 0

    while leftIndex < len(left) and rightIndex < len(right):
        leftValue = sortValue(left[leftIndex], key)
        rightValue = sortValue(right[rightIndex], key)

        if reverse:
            shouldTakeLeft = leftValue >= rightValue
        else:
            shouldTakeLeft = leftValue <= rightValue

        if shouldTakeLeft:
            result.append(left[leftIndex])
            leftIndex += 1
        else:
            result.append(right[rightIndex])
            rightIndex += 1

    result.extend(left[leftIndex:])
    result.extend(right[rightIndex:])
    return result

def mergeSort(data, key, reverse=False):
    if len(data) <= 1:
        return list(data)

    middle = len(data) // 2
    left = mergeSort(data[:middle], key, reverse)
    right = mergeSort(data[middle:], key, reverse)
    return merge(left, right, key, reverse)

def recursiveBinarySearch(data, targetName, left=0, right=None):
    if right is None:
        right = len(data) - 1

```

```

    if left > right:
        return None

    middle = (left + right) // 2
    middleName = normalizeName(data[middle]["name"])

    if middleName == targetName:
        return data[middle]

    if targetName < middleName:
        return recursiveBinarySearch(data, targetName, left, middle - 1)

    return recursiveBinarySearch(data, targetName, middle + 1, right)

def recursivePartialNameSearch(data, keyword, index=0, matches=None):
    if matches is None:
        matches = list()

    if index == len(data):
        return matches

    if keyword in normalizeName(data[index]["name"]):
        matches.append(data[index])

    return recursivePartialNameSearch(data, keyword, index + 1, matches)

# =====
# CREATE (ADD STUDENT)
# =====
def addStudent():
    student = dict()

    name = input("Student Full Name: ").strip()
    grades = inputGrades(name)

    recordUndo("Add Student")
    markDataChanged()

    student["name"] = name
    student["grades"] = grades

    studentData.append(student)

```

```

    print("Student Added")

# =====
# READ (SHOW STUDENT)
# =====
def showStudents(data):
    if not data:
        print("\nNo student data.")
        return

    linkedList = StudentLinkedList.fromList(data)

    print("\n===== ALL STUDENT LIST =====")

    for index, student in enumerate(linkedList, start=1):
        if "avg" in student:
            print(
                f"{index}. {student['name']} - {student['grades']} | "
                f"AVG: {student['avg']:.2f}"
            )
        else:
            print(f"{index}. {student['name']} - {student['grades']}")

# =====
# UPDATE
# =====
def updateStudent():
    showStudents(studentData)

    if not studentData:
        return

    index = inputStudentIndex("Select student index to update: ")

    if 0 <= index < len(studentData):
        student = studentData[index]

        print(f"\nSelected Student: {student['name']}")
        print("""
===== UPDATE MENU =====
1. Update Name Only
2. Update One Subject Grade
3. Clear One Subject Grade

```

4. Replace All Grades

5. Cancel

""")

```
choice = input("Choose update option: ")

if choice == "1":
    newName = input("New Name: ").strip()

    recordUndo("Update Student Name")
    markDataChanged()

    student["name"] = newName
    print("Student Name Updated")

elif choice == "2":
    showSubjectGrades(student)
    subjectIndex = inputStudentIndex("Select subject index to update: ")

    if 0 <= subjectIndex < len(SUBJECTS):
        newGrade = inputGrade(f"New {SUBJECTS[subjectIndex]} Grade: ")

        recordUndo("Update Subject Grade")
        markDataChanged()

        grades = ensureGradeSlots(student)
        grades[subjectIndex] = newGrade
        print("Subject Grade Updated")
    else:
        print("Invalid Subject Index")

elif choice == "3":
    showSubjectGrades(student)
    subjectIndex = inputStudentIndex("Select subject index to clear: ")

    if 0 <= subjectIndex < len(SUBJECTS):
        recordUndo("Clear Subject Grade")
        markDataChanged()

        grades = ensureGradeSlots(student)
        grades[subjectIndex] = 0
        print("Subject Grade Cleared")
    else:
        print("Invalid Subject Index")
```

```

        elif choice == "4":
            newGrades = inputGrades(student["name"])

            recordUndo("Replace All Grades")
            markDataChanged()

            student["grades"] = newGrades
            print("All Grades Updated")

        elif choice == "5":
            print("Update Cancelled")

        else:
            print("Invalid Option")
    else:
        print("Invalid Index")

# =====
# DELETE
# =====
def deleteStudent():
    showStudents(studentData)

    if not studentData:
        return

    index = inputStudentIndex("Select student index to delete: ")

    if 0 <= index < len(studentData):
        recordUndo("Delete Student")
        markDataChanged()

        studentData.pop(index)
        print("Student Deleted")
    else:
        print("Invalid Index")

# =====
# PROCESS DATA
# =====
def processData(data):
    global canSave
    validStudentData.clear()

```



```

processQueue = Queue()
linkedList = StudentLinkedList.fromList(data)
seenNames = set()
hasInvalidData = False

for student in linkedList:
    processQueue.enqueue(student)

while not processQueue.isEmpty():
    student = processQueue.dequeue()
    name = str(student.get("name", "")).strip()
    grades = student.get("grades", list())
    normalizedName = normalizeName(name)
    invalidStudent = False

    if not isValidName(name):
        print(f"Invalid Name: {name}")
        invalidStudent = True

    if normalizedName in seenNames:
        print(f"Duplicate Name: {name}")
        invalidStudent = True
    else:
        seenNames.add(normalizedName)

    if len(grades) != len(SUBJECTS):
        print(f"Invalid Grade Count ({name})")
        invalidStudent = True

    invalidGrades = recursiveInvalidGrades(grades)

    for subjectIndex, grade in invalidGrades:
        if subjectIndex < len(SUBJECTS):
            subjectName = SUBJECTS[subjectIndex]
        else:
            subjectName = f"Subject {subjectIndex + 1}"

        print(f"Invalid Grade ({name} - {subjectName}): {grade}")
        invalidStudent = True

    if invalidStudent:
        hasInvalidData = True
        continue

```

```

        sumGrade = recursiveSum(grades)
        minGrade, maxGrade = recursiveMinMax(grades)
        avgGrade = sumGrade / len(grades)

        validStudentData.append({
            "name": name,
            "grades": grades,
            "sum": sumGrade,
            "avg": avgGrade,
            "max": maxGrade,
            "min": minGrade
        })

    canSave = not hasInvalidData
    print("Data Has Been Processed!")

# =====
# RANKING
# =====
def showRankings(data):
    if canSave:
        if not data:
            print("No processed data. Run Process Data first.")
            return

        sortedData = mergeSort(data, key=lambda student: student["avg"],
reverse=True)

        print("\n==== STUDENT RANKING =====")

        for index, student in enumerate(sortedData, start=1):
            print(
                f"Rank {index} | "
                f"{student['name']} | "
                f"AVG: {student['avg']:.2f} | "
                f"MAX: {student['max']} | "
                f"MIN: {student['min']}"
            )
        else:
            print("Process Data Before Show Rankings")

# =====
# SEARCHING

```

```

# =====
def searchStudent():
    if not studentData:
        print("No student data.")
        return

    keyword = normalizeName(input("Search student name keyword: "))

    if keyword == "":
        print("Search keyword cannot be empty.")
        return

    sortedData = mergeSort(studentData, key=lambda student: student["name"])
    matches = recursivePartialNameSearch(sortedData, keyword)

    if not matches:
        print("Student Not Found")
        return

    if len(matches) == 1:
        showStudentDetail(matches[0])
        return

    print("\n==== MATCHING STUDENTS =====")

    for index, student in enumerate(matches, start=1):
        print(f"{index}. {student['name']} - {student['grades']}")

    selectedIndex = inputStudentIndex("Select result index to show detail: ")

    if 0 <= selectedIndex < len(matches):
        showStudentDetail(matches[selectedIndex])
    else:
        print("Invalid Result Index")

# =====
# UNDO WITH STACK
# =====
def undoLastAction():
    global studentData, validStudentData, canSave
    previousState = undoStack.pop()

    if previousState is None:
        print("No action to undo.")

```

```

        return

    studentData = previousState["studentData"]
    validStudentData = previousState["validStudentData"]
    canSave = previousState["canSave"]

    print(f"Undo Success: {previousState['action']}")

# =====
# SAVE FILE
# =====
def saveFile():
    if canSave:
        with open(DATA_FILE, "w") as file:
            json.dump(studentData, file, indent=4)

        print("Data Saved")
    else:
        print("Process Data Before Saving")

# =====
# LOAD FILE
# =====
def loadFile():
    global studentData

    try:
        with open(DATA_FILE, "r") as file:
            loadedData = json.load(file)

            recordUndo("Load File")
            studentData = loadedData
            markDataChanged()

            print("Data Loaded")
    except FileNotFoundError:
        print("File Not Found")
    except json.JSONDecodeError:
        print("File Is Not Valid JSON")

# =====
# EXPORT FILE

```

```

# =====
def exportPDF(data):
    if not canSave:
        print("Process Data Before Exporting")
        return

    if not data:
        print("No processed data. Run Process Data first.")
        return

    if FPDF is None:
        print("FPDF package is not installed.")
        return

    sortedData = mergeSort(data, key=lambda student: student["avg"],
reverse=True)

    pdf = FPDF()
    pdf.add_page()

    pdf.set_font("Arial", "B", 16)
    pdf.cell(0, 10, "Student Report", ln=True, align="C")
    pdf.ln(5)

    pdf.set_font("Arial", "B", 10)

    pdf.cell(15, 10, "Rank", 1)
    pdf.cell(35, 10, "Name", 1)

    for subject in SUBJECTS:
        pdf.cell(18, 10, subject, 1)

    pdf.cell(20, 10, "Avg", 1)
    pdf.cell(15, 10, "Max", 1)
    pdf.cell(15, 10, "Min", 1)

    pdf.ln()
    pdf.set_font("Arial", "", 10)

    for index, student in enumerate(sortedData, start=1):
        pdf.cell(15, 10, str(index), 1)
        pdf.cell(35, 10, student["name"][:16], 1)

        for grade in student["grades"]:
            pdf.cell(18, 10, str(grade), 1)

```

```

pdf.cell(20, 10, f"{student['avg']:.2f}", 1)
pdf.cell(15, 10, str(student["max"]), 1)
pdf.cell(15, 10, str(student["min"]), 1)
pdf.ln()

if pdf.get_y() > 270:
    pdf.add_page()

pdf.output(REPORT_FILE)
print("PDF exported!")

# =====
# MENU
# =====
def menu():
    while True:
        print("""
===== STUDENT REPORT SYSTEM =====
1. Add Student
2. Show Students
3. Update Student
4. Delete Student
5. Process Data
6. Show Ranking
7. Save File
8. Load File
9. Export PDF
10. Search Student
11. Undo Last Change
12. Exit
""")

        choice = input("Choose: ")

        if choice == "1":
            addStudent()
        elif choice == "2":
            showStudents(studentData)
        elif choice == "3":
            updateStudent()
        elif choice == "4":
            deleteStudent()
        elif choice == "5":

```

```
        processData(studentData)
    elif choice == "6":
        showRankings(validStudentData)
    elif choice == "7":
        saveFile()
    elif choice == "8":
        loadFile()
    elif choice == "9":
        exportPDF(validStudentData)
    elif choice == "10":
        searchStudent()
    elif choice == "11":
        undoLastAction()
    elif choice == "12":
        break
    else:
        print("Invalid Option")

if __name__ == "__main__":
    menu()
```